

Preservation and Analysis of a Legacy DBL Accounting System

A Technical Report

Second edition

Noah Parsons

25 August 2026

Abstract

This report documents the preservation, verification and analysis of a multi-module accounting system written in the DBL programming language and in continuous production use from 1980 until 2026. The source code was copied from a single desktop computer, audited for confidential information, and placed under version control in a private repository. The archive was then verified by rebuilding part of the system from that repository using the original 1991 compiler and comparing the result against the software running in production. The two were found to be identical apart from embedded date stamps. Analysis established the size of the system, its authorship, the computer platforms it has run on, the structure of its data, and the functions it performed. The original developer has been identified as Custom Computer Services, a company that no longer exists, and this identification has been confirmed by the system's operator. The corporate ownership of the copyright remains unresolved.

Note on this edition. This is a revised report. Five conclusions in the first edition have been corrected in the light of evidence found later, and each correction is identified where it occurs. The corrections are collected in Section 12 for convenience.

Contents

1	Introduction	2
1.1	Background	2
1.2	Objectives	2
1.3	A note on terminology	2
2	The System as Received	3
2.1	Inventory	3
2.2	Third party components	3
3	Protection of Confidential Information	3
3.1	Approach	3
3.2	Findings	3
3.3	Verification	4
4	Establishment of the Repository	4

5	Verification of Completeness	5
5.1	Coverage	5
5.2	Reconstruction	5
5.3	Comparison against the production build	5
5.4	Significance of this result	5
6	Analysis of the Source Code	6
6.1	Scale	6
6.2	Authorship	6
6.3	Dates and the pace of change	7
6.4	Computer platforms	7
6.5	Optional modules	7
6.6	Data structures	8
6.7	The shared subroutine library	9
6.8	Dependencies between programs and data	9
6.9	Navigation and functions	10
6.10	Screen text, reference data and error handling	11
6.11	An undocumented program	11
7	Analysis of the Compiled Programs and Runtime	11
7.1	The startup banner	11
7.2	Further findings from the runtime	11
7.3	The entry point	12
7.4	Two stray files	12
8	Analysis of the Data Files	12
8.1	The data export format	12
8.2	Index definitions	12
8.3	Independent validation of the schema	13
9	Identification of the Software's Origin	13
9.1	The question	13
9.2	Nature of the product	13
9.3	The identification	13
9.4	Corporate records	13
9.5	Dating the choice of tools	14
10	Correspondence	14
10.1	The system's operator	14
10.2	The compiler's present owner	14
11	Ownership and Licensing	15
12	Corrections to the First Edition	15
13	Deliverables	16
14	Outstanding Matters	16
14.1	Immediate	16
14.2	In progress	16
14.3	Further work	16
15	Conclusion	17

1 Introduction

1.1 Background

The subject of this report is an accounting system used by a small accounting practice. The practice is being wound up, and its operator has given written permission for the software to be preserved and developed. At the start of this work the software had one surviving copy, held on a single desktop computer.

The system is written in DBL, a business programming language derived from Digital Equipment Corporation's DIBOL. DBL was developed by Digital Information Systems Corporation, a company renamed Synergex in 1996 and still trading. The compiler used to build the system is DBL version 4.41 for MS-DOS, released in October 1991.

The oldest comments in the source are dated 1980. The system remained in use until 2026, a service life of approximately forty six years.

1.2 Objectives

1. To create a second copy of the source code, held somewhere other than the original computer.
2. To ensure that no confidential information belonging to the practice or its clients was included in that copy.
3. To establish whether the copy was complete, meaning whether it contained everything necessary to reproduce a working system.
4. To record what could be determined about the software's origins while the person who operated it was still available to be asked.
5. To understand the system's structure and function well enough that the knowledge would survive independently of the software itself.

All five have been met. The fourth produced a definite answer, which was not expected at the outset.

1.3 A note on terminology

This report is intended to be readable without prior knowledge of software development. The following terms are used throughout.

Source code The human readable instructions written by a programmer.

Compiler A program that translates source code into a form the computer can execute.

Runtime A program that executes the translated result. In this system the compiler is `DBL.EXE` and the runtime is `DBR.EXE`.

Repository A managed store of files recording every change made to them over time. The repository used here is private, meaning visible only to its owner.

Record definition A description of the layout of stored data, listing each field, its name and its size.

ISAM Indexed sequential access method, the file format used for the system's data. Each file holds fixed length records with an index that allows a record to be found by its key.

2 The System as Received

2.1 Inventory

The software was received as a directory tree of 1,922 files. A copy was made and all subsequent work carried out on the copy.

Directory	Contents	Files	Size
SRC/	Source code	1,002	6.1 MB
DAT/	Business data for five companies	244	50 MB
EXC/	Compiled programs	523	12 MB
RUN/	Configuration and reference data	74	391 KB
TMP/	Working files	36	1.2 MB
DBL/	The DBL compiler and runtime	31	1.4 MB
LIB/	Backup and restore scripts	6	6 KB
BCK/	Backup directory	0	empty
SPL/	Print spool directory	0	empty

Table 1: Contents of the directory tree as received.

2.2 Third party components

Two components belong to parties other than the practice and were excluded from the repository.

The DBL/ directory contains the DBL development system: the compiler, linker, librarian and runtime, with their supporting libraries. This is a commercial product of Digital Information Systems Corporation.

The second was identified only later. Two scripts in RUN/ load screen mode files from a directory named C:\FANSI, which indicates that FANSI-CONSOLE, a commercial terminal driver for MS-DOS, was installed on the production machine. It is not present in the archive. The source refers to it by name in a conditional compilation setting, so the software was written with that specific driver in mind.

3 Protection of Confidential Information

3.1 Approach

Before any file was placed in the repository, the tree was examined for personal and business information.

Two principles were adopted. First, the audit had to be complete before the first entry was recorded, because a repository retains a permanent history and a file added then removed remains recoverable. Second, the rules controlling what enters the repository were written as a list of permitted items rather than forbidden ones, so that anything unexpected is excluded by default.

3.2 Findings

Four categories of confidential material were identified.

Live business records. The DAT/ directory holds the financial records of five client companies in five numbered subdirectories, including customer masters, vendor masters and payroll files.

Excluded entirely.

Working files named after clients. The TMP/ directory contains files whose names appear to be client surnames, and annual audit working files spanning 2014 to 2025. Excluded entirely.

Stored passwords. The RUN/ directory, which otherwise appears to hold ordinary configuration, contains the system's security file. Analysis of the source confirmed that this file stores user passwords. Excluded, and its presence caused the whole directory to be withheld pending a file by file review.

Payroll records outside the data directory. During that review, one file was found to contain the names, addresses and Social Security numbers of 269 employees. Every other file of the same type contained no such information. Excluded, and all files of that type were then excluded as a precaution.

The last finding is the most significant. It demonstrates that the location and naming of a file are not reliable indicators of its contents, and it means the practice's confidential data extends beyond the directory that obviously holds it. This has been reported to the operator, since it affects how the original computer should be disposed of.

3.3 Verification

Three checks were performed.

The names of all files staged for inclusion were examined, confirming that no data files, security files, form files or logs were present. The contents of those same files were then searched for text matching the format of a Social Security number, with no matches. Finally the entire tree was searched, including excluded directories, to establish whether further material lay outside the areas already identified. Two files matched, both already excluded.

That search would not detect Social Security numbers stored without punctuation, and the record definitions later confirmed that the system stores them in exactly that form. The search therefore establishes that the repository is clean but does not measure the extent of data in the excluded directories.

4 Establishment of the Repository

The audited material was committed to a private repository in two stages: 1,003 files comprising the source, the backup scripts and the top level configuration, followed by 63 files from RUN/ after review. The latter comprise device mapping tables, parameter files, and federal withholding tax tables covering 1990 to 2026.

Two configuration files preserve the material's integrity. The first defines the permitted list, with rules written in both letter cases because they are treated as case sensitive on some operating systems and not others. The second preserves the MS-DOS line ending convention, without which the files would be altered on retrieval in a way that would prevent the software being rebuilt.

Nine files use a different line ending convention from the remainder. These are clustered around the programs that produce tax forms, and are consistent with the evidence in Section 6.3 that the system was maintained on a Unix computer.

One file in the repository, AUTOEXEC.BAT, was examined and found to be Microsoft's standard template for running MS-DOS programs under Windows. It is not part of the accounting system and appears to have been copied into the directory during the software's time on a Windows computer.

5 Verification of Completeness

5.1 Coverage

The production system contained 519 compiled programs. The repository contains 769 source files. Comparing the two established that **every compiled program in production has its source in the repository**.

The 250 source files with no corresponding compiled program fall into two groups: the 76 modules of the shared subroutine library, which combine into a single library file rather than compiling individually, and modules belonging to optional features never installed at this practice. The repository therefore contains more of the product than the practice used.

5.2 Reconstruction

A virtual computer was created running FreeDOS. A fresh copy of the repository was retrieved, combined with the DBL compiler from the original tree, and transferred in.

Three obstacles were encountered, all properties of the environment rather than faults in the archived source:

1. The system's setup script defines approximately thirty environment settings but does not make the compiler itself reachable.
2. The default memory reserved for environment settings under FreeDOS was too small to hold all thirty, and later settings were being silently discarded.
3. The system requires a standard MS-DOS file locking utility, which was not loaded by default.

Once addressed, the build completed without error. All 76 library modules compiled, the library was assembled, and the system module compiled and linked against it. The resulting program started, initialised, and stopped at its own error handler when it could not open its first data file, which is the expected outcome given that the data was deliberately excluded.

The exercise was later repeated under Windows XP, where the system ran to its full opening screens. This is described in Section 7.

5.3 Comparison against the production build

The library produced by this process was compared byte by byte against the library running in production.

The two files are the same size. Of 184,723 bytes, 1,246 differ, which is 0.67 per cent. The differences occur in clusters of nine bytes at the positions where the internal headers of the library's component modules are stored.

Every differing byte in the entire file is one of twelve characters: A, J, g, l, and the digits 0, 1, 2, 3, 4, 5, 7 and 9. These spell the abbreviations for August and July together with the digits of a date and time.

The two libraries are identical apart from the date on which each was built.

5.4 Significance of this result

Two claims should be distinguished.

The determinism of the compiler is not itself remarkable. A single pass compiler of that era has few opportunities to introduce variation. Modern build systems are difficult to reproduce exactly because of features that did not exist in 1991.

What the comparison establishes is that the source in the archive is the source from which the running system was built. Had anyone modified the production software directly without recording the change, or had the archived source drifted from the deployed version, the files would differ in more than their dates. This is of particular interest given the finding in Section 6.2 that the operator has continued to modify the programs without documenting it: those modifications were properly compiled into what was running.

It also dates the production build. Because the two match, the production library must itself have been built with DBL 4.41, and therefore no earlier than October 1991.

6 Analysis of the Source Code

6.1 Scale

Code	Subsystem	Files	Lines
PR	Payroll	202	47,796
AR	Accounts receivable	184	37,043
AP	Accounts payable	134	30,808
GL	General ledger	120	30,437
XC	Shared subroutine library	76	7,213
SY	System functions	34	4,347
SC	Security	15	2,989
BR	Bank reconciliation	10	1,340

Table 2: Size of each subsystem. Total 161,973 lines.

Comments account for 3.2 per cent of lines. Programs range from 11 to 960 lines.

6.2 Authorship

Approximately one third of the source files, 245 of 776, carry a comment recording who wrote the program. Two conventions are used: one records initials and a date, the other a full name without a date.

Analysis by subsystem reveals three groups of authors with almost no overlap.

Group	Subsystems	Authors
A	Payroll, shared library, security, bank reconciliation	WBK, CJS, LWB, JLS
B	Accounts payable, accounts receivable, system	David Hawkinson, Dan Roesler
C	General ledger	Deb Haffner, DER

Table 3: Author groups by subsystem.

Six individuals are named in full: Loren Bergeson, who also appears as LWB and is the principal author of the shared library; David Hawkinson; Dan Roesler; Deb Haffner; Darin Capp; and Jackie Soule. Eight sets of initials remain unmatched, of which three appear only as later modifiers.

Correction to the first edition. That report inferred from this division that the modules had been acquired from three separate suppliers. The operator has since confirmed that all three groups were employees of a single company. The division reflects internal team structure, not separate origins.

6.3 Dates and the pace of change

Of 106 datable comments: 1980 (83), 1983 (12), 1981 (3), 1982 (3), 1985 (2), 1986 (2), 1984 (1). One individual, WBK, modified substantially more files than he wrote, indicating a long term maintenance role.

Correction to the first edition. That report concluded from these dates that the system was written in 1980, maintained for six years, then frozen. The operator has since stated that she has continued to modify the programs herself through the present year without recording it in the comments. The dated comments therefore reflect the period during which the original development company maintained a documentation convention, not the period during which the software changed. The system is not a frozen artefact but living code with more than four decades of undocumented modification.

6.4 Computer platforms

The source contains instructions that include or exclude sections of a program depending on the target computer.

Setting	Occurrences	Platform
UNIX	1,946	Unix operating systems
RT11	1,692	DEC RT-11, for the PDP-11 minicomputer
VAX	1,225	DEC VAX minicomputers
RSTS	8	DEC RSTS/E, for the PDP-11
MSDOS	1	Microsoft MS-DOS

Table 4: Platform specific code, by frequency.

604 of approximately 990 source files, or 60 per cent, contain at least one such instruction. This is portable code rather than a program written for one machine.

The three leading platforms occur at similar frequencies, indicating parallel maintenance rather than succession. The single occurrence of MS-DOS is instructive: the port made MS-DOS the unmarked default, so that code not explicitly assigned elsewhere is the MS-DOS version. Consistent with this, a build script survives in both Unix and MS-DOS forms. The build was ported rather than the source.

The operator’s account adds machines the code does not name. In her words the progression was RT-11, then Convergent Technologies S120 and S80, then an IBM RS/6000, then personal computers. The RS/6000 runs AIX, which is the probable origin of the 1,946 Unix settings. She also recalls an attempt to run the system on Linux. Two further platforms are evidenced by the runtime, discussed in Section 7.

A program in the bank reconciliation module widens a date field to four digits, which is work associated with the year 2000.

6.5 Optional modules

A second group of settings controls optional features rather than platforms.

Setting	Occurrences	Apparent meaning
JCST	150	Job cost variant
ECST	95	Cost variant
UCST	92	Cost variant
BATCH	76	Batch processing
BNKREC	60	Bank reconciliation module
BANK	50	Banking module
SCTRM	33	Terminal based screen handling
SCCMP	32	Compiled screen handling
CERT	8	Certification
COMLOG	6	Logging

Table 5: Optional feature settings found in the source.

These cause whole modules to be included in or omitted from a build. A single body of source that assembles into several different configurations indicates software sold as a package with selectable modules.

Two further observations support this. The main menu offers an Inventory option for which no menu program exists anywhere in the source, indicating a module advertised but not installed. The setup script defines directory locations for three further modules, covering job analysis, job cost and order entry, none of which were installed.

The prominence of the banking settings is now explained. The operator states that the system began life as a package for banks and was later extended to commercial accounting. This also accounts for bank reconciliation sharing its authors with the payroll and the shared library: those were the original product.

Correction to the first edition. That report treated these settings as evidence that the software was purchased from an outside supplier. They are evidence that it was a configurable product, which is confirmed, but the supplier developed it in house rather than assembling it from acquired parts.

6.6 Data structures

The system’s data layouts are held in 160 files. A parsing program was written to extract them into a single document.

The parser was developed in three iterations. The first handled 62 per cent of the input. Examining the failures revealed three features not anticipated: the companion files declare layouts as shared storage rather than local records; fields may be declared as arrays with a repeat count; and some declarations carry literal initial values. The final version leaves two lines of 160 files unhandled.

The result describes 339 records containing 2,425 fields, with the byte position of every field computed from the sizes of those preceding it.

Four observations are of practical importance to any migration.

Implied decimal points are recorded separately from the field declaration. A field may be declared as seven digits while a comment records that four fall after the decimal point. A migration reading only the declaration would misinterpret such a value by a factor of ten thousand.

Records may be redefined. The same storage may be described several times over, with the correct interpretation selected at run time by a code in the first byte.

Social Security numbers are stored as nine digits without punctuation.

One layout is an eighty column punched card image. Its redefinitions each begin with a card code field. This is the format in which transactions were originally submitted, and it survived every change of platform unaltered.

A further file constructs terminal control sequences as a data record, with screen row and column held as writable fields. This is how the system positions text on screen.

6.7 The shared subroutine library

The system contains 14,503 calls to shared subroutines, approximately fifteen per file.

Routine	Calls	Apparent function
ABRTP	1,394	Abort with a message
CLRSC	1,323	Clear the screen
OPENP	1,292	Open a file with protection
INPTD	1,133	Accept a numeric input
CLOSP	906	Close a protected file
WRITS	804	Write a record
READ	775	Read a record
INPTA	682	Accept an alphabetic input
INUSE	631	Test whether a record is in use

Table 6: Most frequently called shared routines.

All nine concern either the screen or protected file access, indicating a strict common structure followed by every program. That uniformity is the most probable explanation for how code written by at least a dozen people across five platforms remained coherent over four decades.

Of the 76 routines, approximately 22 are never called. These include the entire family of date arithmetic routines, both check digit routines, and a series of seven numbered file handling routines of which only the first is used. They are compiled into the library on every build regardless.

6.8 Dependencies between programs and data

Extracting the declarations by which programs request record definitions produced 2,567 relationships.

The distribution is uneven. Two definitions are near universal: system wide settings, used by 625 programs, and date definitions, used by 524. The next most used appears in only 77. There is no third universal dependency, and each subsystem otherwise confines itself to its own data. This indicates loosely connected subsystems that could be replaced individually.

Examining which definitions are shared between subsystems identifies the points at which modules connect. After the two universal definitions and the company file, used by six of eight subsystems, the shared items are almost entirely those of the general ledger. The general ledger is the hub, and the interface file through which other modules post entries to it is the most important connection point in the system.

Three job cost definitions are shared across subsystems despite the job cost module never having been installed. The means by which other modules would post to job costing are present regardless of whether the module was purchased.

6.9 Navigation and functions

The mechanism by which one program starts another was identified: rather than calling another program and waiting, a program ends itself and names its successor. There are 2,447 such transitions.

Each is written once for each platform, using the directory names defined by the setup script. This is the purpose of the thirty environment settings. The transitions are also affected by the optional modules: on leaving payroll, the system returns to the main menu or proceeds to the banking module depending on which was installed.

The most frequent destinations are the subsystem menus. One further destination is prominent: a general purpose sorting utility, the target of 230 transitions, which is not present in the source. It is presumed part of the DBL runtime, and any replacement would need to supply an equivalent.

The text of the menus was recovered from the programs that display them.

Module	Functions offered
General ledger	Account maintenance, transaction entry, trial balance, source cross reference, accounting period maintenance, financial statements, interface file maintenance, automatic transactions, profit centres, consolidated ledger, compression of the year to date file, year end processing
Accounts payable	Due date changes, payment preparation, partial payments, check printing, prepayments, miscellaneous vendors, check history, adjustments, printing and clearing of tax form information
Accounts receivable	Customer maintenance and enquiry, sales entry, cash receipts, open credit application, account enquiry, customer listings, aged trial balance, statements, monthly sales and cash summaries, sales analysis, sales tax reporting, finance charge calculation, purging and clearing
Payroll	Employee master maintenance, on demand applications, transaction entry, payroll calculation, check printing, reporting, manual transactions, quarterly returns, year end tax forms, field changes, file reorganisation, history, backup and restore
Bank reconciliation	Entry and reversal of cleared checks, edit lists, deletion, listings, bulk addition and clearing of records
Maintenance	File protection clearing, security maintenance, company file maintenance, spool file printing, header file maintenance, address maintenance, backup, restore, program startup

Table 7: Functions offered by the system.

6.10 Screen text, reference data and error handling

The system displays 929 distinct pieces of text. The most frequently repeated form a single sequence used across every module: enter a new transaction, change an existing transaction, delete an existing transaction, print an edit list, and post to the main file. A common workflow applied uniformly explains how one operator could move between modules without retraining.

The tax tables hold one file per year from 1990 to 2026, each of approximately sixteen rows of eighteen digits. The trailing digits of the first row of the 2026 file correspond to a rate of 37.0 per cent; the 1990 equivalent is 28 per cent. These match the top federal marginal rates for those years. The format has remained unchanged across thirty six tax years and five platforms.

Every one of the twenty most frequent messages passed to the abort routine concerns a failure to open a file. The system contains 1,159 explicit checks of whether an operation succeeded. Its failure model is accordingly simple: if a file cannot be opened, report which file and stop.

Two of the abort routines display the message *Call Your Software Representative*. This phrasing indicates software supplied to a customer by an identified supplier, rather than written by the organisation that uses it.

6.11 An undocumented program

The system directory contains a program which draws a target moving across the top of the screen and a gun sweeping along the bottom, and awards points for intercepting one with the other. It is referenced by no menu and by no other program, and can be reached only by typing its name into the Program Startup option. It is nonetheless listed in both build scripts and was compiled into every release.

It is recorded here because it is part of the archive and because it indicates that whoever maintained the system had unrestricted access to it rather than receiving a sealed product.

7 Analysis of the Compiled Programs and Runtime

The first edition examined only the source. Examination of the compiled programs and the vendor's runtime subsequently produced the most direct evidence in the investigation.

7.1 The startup banner

When the system starts, the DBL runtime prints an ownership notice. Recovered in full, it reads that DBL is the copyrighted, unpublished work of Digital Information Systems Corporation, followed by the words *It has been licensed to* and then three stored values:

Licensee	Custom Computers Service
Customer identifier	1614
System serial	65110f1723

This is the only company name recorded anywhere in the system. Its significance is discussed in Section 9.

7.2 Further findings from the runtime

The runtime identifies itself as the Synergy/NetWare edition, establishing that the installation ran multi-user on a Novell network with terminals rather than on a single computer. This is a sixth platform, and one the source only hints at.

The runtime also records the developer’s address and telephone number at the time: 11070 White Rock Road, Rancho Cordova, California, and (916) 635-7300. Rancho Cordova adjoins Gold River, where the successor company trades today.

A third party communications library dated 1984 and 1985 is embedded in the runtime.

7.3 The entry point

The program that starts when the system is launched is not the main menu. The launch script calls a second script, which runs a company control program. This displays a screen headed *Company File Control* asking which client company is to be processed. The multi client structure is therefore the first thing an operator encounters.

7.4 Two stray files

Two files unrelated to the accounting system were found in the working directory: a command interpreter from Windows 98, and a CD-ROM driver of the kind supplied on Windows 98 startup disks. These evidence a Windows 98 period between the MS-DOS deployment and the Windows XP machine now in use, and at least one machine migration performed from a startup disk.

8 Analysis of the Data Files

One file in the configuration directory, initially assumed to be an activity log, proved to be a captured transcript of a maintenance session in which data files were rebuilt. It contains no client data, and it supplies two things the source does not.

8.1 The data export format

The transcript shows the vendor’s utility unloading an indexed file into a flat sequential file, then reloading it. This establishes that the flat format is a supported export produced by a supplied tool.

The practical consequence is that **a migration path already exists**. Each indexed file can be unloaded to a flat file of fixed length records, which can then be read using the field positions recorded in the extracted schema. It is not necessary to reverse engineer the indexed file format.

8.2 Index definitions

The transcript records how each file is indexed, which the source does not describe.

File	Contents	Record	Key	Position
AR01CUSM	Customer master	408	6	1
AR01CUSI	Customer index	31	31	1
AR01ADRM	Address master	31	5	1
AR01CATM	Category master	152	7	1
PR01PB05	Payroll	1104	4	1

Table 8: Index definitions recovered from the maintenance transcript. Lengths in bytes.

Every file uses a single key beginning at the first byte, with duplicates disallowed and ascending order. A replacement system could use an ordinary primary key on each table.

8.3 Independent validation of the schema

The record lengths in the transcript provide an independent check on the parsing program, which derived its figures from the source alone. Two match exactly.

One did not. The customer master appears as 408 bytes in the transcript against 313 computed from the source. The discrepancy was resolved by measuring the exported file itself, which is 344,925 bytes. That divides exactly by 315, being 313 bytes of record plus a two byte line terminator, giving 1,095 records. The source and the parser are therefore correct and the transcript entry reflects a different revision of the layout.

This also gives the first measure of the practice's scale: 1,095 customers on the first company.

9 Identification of the Software's Origin

9.1 The question

The first edition could not identify who wrote the software. No copyright notice, company name or statement of rights appears anywhere in the 990 source files. A search of the whole tree, in every case form and spelling, returned no ownership statement of any kind.

9.2 Nature of the product

Examination of the payroll record definitions found a client number field, and the data directory holds five separately numbered companies. The system is therefore client write up software, designed for an accounting practice maintaining the books and payroll of several client businesses. This is a distinct category of product with its own suppliers.

9.3 The identification

The startup banner recorded in Section 7.1 supplied a company name: Custom Computers Service. This was put to the system's operator, who confirmed the following.

- The software was written by Custom Computer Services.
- All of the programs were written in house by that company's employees. The individuals named in the comments, and those identified only by initials, were their staff.
- The system began as a package for banks and was subsequently extended to commercial accounting.
- The practice obtained the software from that company.
- Custom Computer Services was subsequently sold. The purchaser did not succeed with it and the business closed. The company no longer exists.

The company is therefore both the developer and the supplier, which reconciles the apparently conflicting evidence: the software was a configurable product sold to customers, which accounts for the optional module settings and the supplier reference in the abort routine, and it was written in house, which accounts for the internal division of authorship.

9.4 Corporate records

A South Dakota company registration exists for an entity named Custom Computer Services, Incorporated, formed in 1996 and dissolved in 2012, with a principal office in Sioux Falls. It

was initially discounted because the practice is in Rapid City and the date is later than the software.

It may nonetheless be relevant. The operator's account of the company being sold and subsequently failing is consistent with a 1996 incorporation under new ownership followed by a 2012 dissolution. This has been put to her and is unresolved.

Company records in South Dakota prior to 2016 are not fully represented in the online index, so an absence from that index does not establish that an entity never existed. A direct enquiry to the Secretary of State would be required to search the earlier period.

9.5 Dating the choice of tools

Published histories record that DBL was first sold in mid 1979 for the PDP-11 running RT-11 and RSTS/E, with a second version in November 1980. The source contains conditional code for both, and its oldest comments are dated 1980. The authors were therefore among the earliest users of a newly released compiler.

10 Correspondence

Two enquiries were made and both have been answered.

10.1 The system's operator

A set of questions was put to the practice's operator concerning the origin of the modules, the identity of the named authors, the survival of paperwork, the purchase of the compiler and the original hardware. Her replies are incorporated throughout this report and are the source of the identification in Section 9.3 and the corrections in Sections 6.2 and 6.3.

She also granted written permission for a copy of the software to be retained and developed, and approved the making of a backup of the practice's data.

She holds several printed manuals for the application. These have not yet been examined and are expected to be the only surviving record of the product's name and ownership.

10.2 The compiler's present owner

An enquiry was made to Synergex, the successor to Digital Information Systems Corporation, concerning three matters: whether records survive of software houses that licensed DBL in its early years, what licensing exists for an application of this vintage, and whether documentation for version 4 survives.

The response was substantive. Their development team believe they may recognise the application and have asked for the author names to confirm it. They note that some of the original authors may be employed by them now, and that they maintain contact with retired staff.

On licensing, they state that legacy maintenance and modernisation is their principal business, that the language is now at version 12, and that they avoid changes which would prevent older code compiling. They expect a body of code of this age to produce compiler errors of specific and known kinds, chiefly ambiguous symbols, circular references and duplicated routines with subtle differences, but that the syntax and build process are broadly unchanged. They have offered to assess the difficulty of migration, and have offered educational licensing.

On documentation, they are searching their physical archive for a version 4 manual. Documentation for version 6 onwards is published online.

This materially alters the outlook. A supported recompilation under a current release is a bounded and achievable exercise rather than a theoretical one.

11 Ownership and Licensing

This section records the position as understood and is not legal advice.

The software was written by employees of Custom Computer Services in the course of their employment. The copyright therefore vested in that company rather than in the individuals, which is consistent with the operator's statement that no one owns the programs individually.

That company was sold and subsequently ceased to exist. Rights in such circumstances do not lapse; they pass with the sale of the business and remain with whatever entity or creditors survive its dissolution. Establishing where they now rest would require tracing the corporate history described in Section 9.4.

The practice's position is that of a customer. The written permission granted by the operator is effective as between the parties and records her consent, but she cannot transfer rights she does not hold.

The DBL compiler and runtime are the property of Synergex and were excluded from the repository accordingly. The terminal driver identified in Section 2.2 is a separate third party product, also absent.

The materials produced during this work, comprising the parsing program, the extracted schema, the dependency listing and the analysis recorded in this report, are original work and are unaffected by the above.

The practical consequence is that private study, analysis and written description are unaffected, while publication or distribution of the original source would require the ownership question to be resolved. This is why the repository is private.

12 Corrections to the First Edition

For clarity, the five corrections made in this edition are collected here.

1. **Origin of the modules.** The first edition inferred from three distinct author groups that the modules were acquired from separate suppliers. All three were teams within one company.
2. **Dating.** The first edition concluded that the system was frozen after about 1986. The operator has continued to modify it to the present day without documenting the changes.
3. **Copyright notices.** The first edition reported that no ownership notice existed. This was correct for the application source but the vendor's runtime carries a full notice, which was not examined until later. An intermediate revision of this analysis wrongly stated that the compiler also carried such a notice; it does not.
4. **Distribution.** The first edition proposed, on the strength of the supplier reference in the abort routine, that the software was sold through dealers. The developer and the supplier were the same company.
5. **Platform count.** The first edition listed four platforms, then five. The correct figure is at least seven once the operator's account and the runtime edition are included: RT-11,

RSTS/E, VAX, Unix on Convergent and RS/6000 hardware, Novell NetWare, MS-DOS and Windows.

13 Deliverables

Source archive 1,066 files comprising the complete source code, backup scripts, configuration and reference data, and build scripts.

README A description of the system, its platform history, the layout of the repository, the build procedure including the three environment requirements discovered during reconstruction, and an explanation of what was excluded and why.

Provenance record The authorship analysis, platform and module evidence, reconstruction result, completeness result, and outstanding questions.

Data documentation A description of 339 records and 2,425 fields with computed byte positions.

Parsing program The program that generates the above from the source.

Dependency listing 2,567 program to data relationships.

This report.

14 Outstanding Matters

14.1 Immediate

1. **Backup of the business data.** The practice's records exist on the original computer and one copy. They are not in the repository and must not be. A dedicated encrypted drive has been approved by the operator and should be prepared without delay. This concerns the continuity of the business rather than the preservation of the software.
2. **The manuals.** These are expected to identify the product by name and record its ownership. They are held by a business being wound up and are at risk.

14.2 In progress

1. The licence identifiers recorded in Section 7.1 have been supplied to Synergex, together with the author names, in the hope of identifying the application from their records.
2. Synergex are searching for a version 4 manual.
3. The relationship between the 1996 Sioux Falls registration and the original company has been put to the operator.
4. Permission from the operator to share the source with Synergex for a migration assessment has been requested.

14.3 Further work

1. **Detailed reading of individual programs.** The system has been analysed structurally but no single program has been read in full. This is a prerequisite for any reimplementation.

2. **A reader for the data files.** Using the extracted schema and the export format described in Section 8.1, a program could be written to read the system’s data directly. This is the component any migration requires and it is unaffected by the ownership question.
3. **Trial recompilation under a current release**, subject to the licensing discussed in Section 10.2.

15 Conclusion

The objectives set at the outset were met, and the fourth was met more fully than expected.

The source code of a forty six year old accounting system has been copied from a single computer into a managed repository, audited twice for confidential information, and confirmed to contain no client data. The archive has been shown to be complete, in that every program in production has surviving source, and functional, in that it reproduces the production software exactly apart from the date of the build.

The system has been documented to a degree that would allow it to be understood without reference to the software itself. Its scale, structure, data layouts, index definitions, internal dependencies and functions are recorded, and a practical route for extracting its data has been identified.

The developer has been identified. The software was written in house by employees of Custom Computer Services, a company that began by supplying systems to banks, extended into commercial accounting, was later sold, and no longer exists. This was established from a single line printed by the vendor’s runtime at startup, and confirmed by the person who bought the software from them.

Where the copyright now rests is unresolved, and may not be resolvable. The work described here does not depend on the answer.